

Bases de données avancées

Curseurs



Pr. ELALAOUI Hasna
h.elalaoui@uca.ac.ma

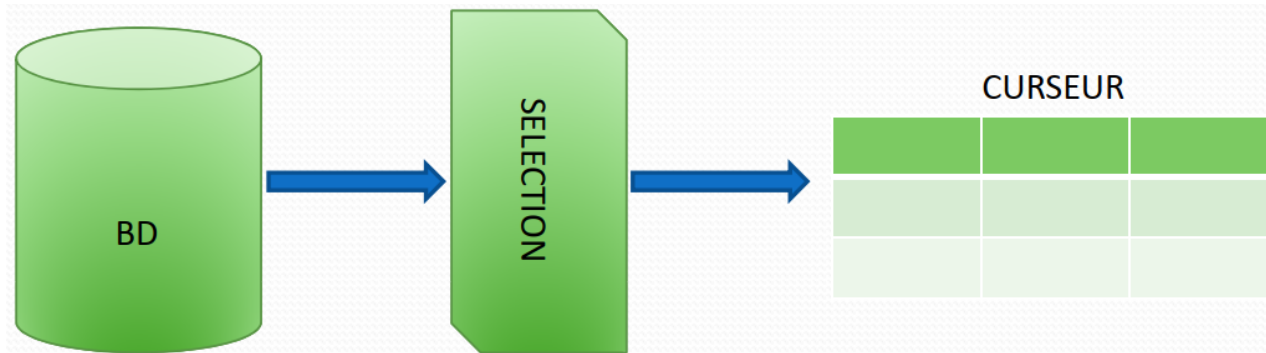
1

Introduction



Problématique

- Les instructions de type **select ... into ...** ne fonctionnent que sur des requêtes ne retournant qu'une unique ligne.
- Ce n'est pas souvent le cas dans toutes les applications Oracle. **Quelle est la solution?**
- Le serveur Oracle utilise des zones de travail appelées Zone Sql Privées pour exécuter les instructions SQL et pour stocker les informations en cours de traitement.
- On peut utiliser des **curseurs** PL/SQL pour nommer une zone SQL privée et accéder aux données qu'elle contient.



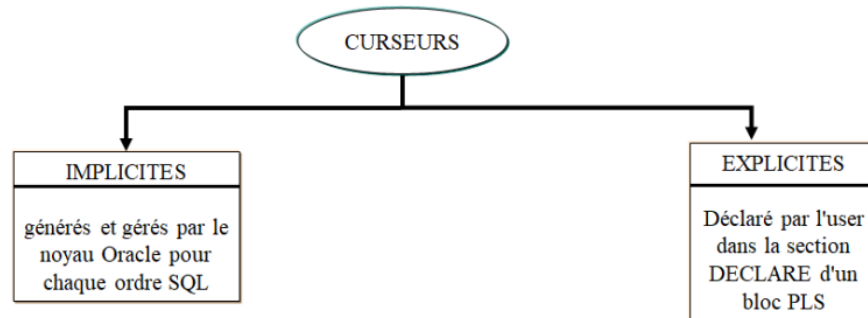


Qu'est ce qu'un curseur?

- Un **curseur** est une zone mémoire qui permet de traiter individuellement chaque ligne renvoyée par un SELECT
- Un **curseur** est une **variable dynamique** qui prend pour valeur le résultat d'une requête **multilignes**.
- Le curseur permet d'accéder aux champs où les commandes SQL sont exécutées et les informations sont stockées

Types de curseurs :

- curseur **implicite** : créé et géré par Oracle à chaque ordre SQL
- curseur **explicite** : créé et géré par l'utilisateur afin de pouvoir traiter un SELECT qui retourne plusieurs lignes





Curseur explicite

- Le curseur explicite stocke les enregistrements résultat d'un « select » et permet de les récupérer un par un pour pouvoir les traiter.
- La récupération se fait par « **FETCH** »
- «FETCH» récupère la ligne courante du curseur et copie les données dans des variables.
- Le traitement à effectuer est à préciser par l'utilisateur.

2

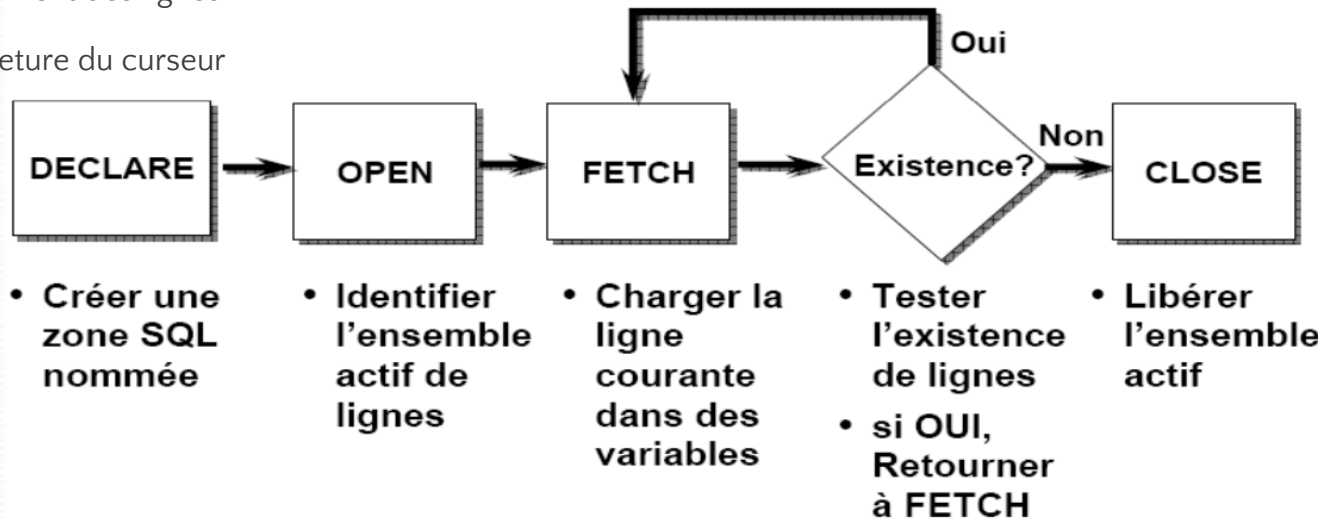
Utilisation d'un curseur



Aperçu général

L'utilisation d'un curseur explicite nécessite 4 étapes :

1. déclaration du curseur
2. ouverture du curseur
3. traitement des lignes
4. fermeture du curseur





Déclaration d'un curseur

- La déclaration du curseur permet de stocker l'ordre Select dans le curseur.
- Le curseur se définit dans la partie DECLARE d'un bloc PL/Sql.

CURSOR nomcurseur **IS** Requete_SELECT ;

Déclaration du curseur
CURSOR ... IS

Syntaxe

```
DECLARE  
  CURSOR <nom_curseur> IS  
  SELECT col1, col2, col3, ...  
  FROM tab1, tab2, tab3, ...  
  WHERE <condition>;  
...;
```

Exemple

```
DECLARE  
  CURSOR c_employe IS  
  SELECT empno, ename, salaire  
  FROM employe  
  WHERE salaire BETWEEN 1000 AND  
  2500 ;
```




Ouverture d'un curseur

- Après avoir déclaré le curseur, il faut l'ouvrir dans la section exécutable (BEGIN) afin de faire exécuter l'ordre SELECT : **OPEN** nom_curseur ;
- L'ouverture du curseur réalise :
 - l'allocation mémoire du curseur
 - l'analyse sémantique et syntaxique de l'ordre
- **C'est seulement à l'ouverture du curseur que la requête SQL s'exécute.**

```
Declare  
Cursor DEPT10 is  
select ename, sal from emp where deptno=10 order by sal ;  
Begin  
Open DEPT10;  
.....  
End ;
```



Traitement des lignes



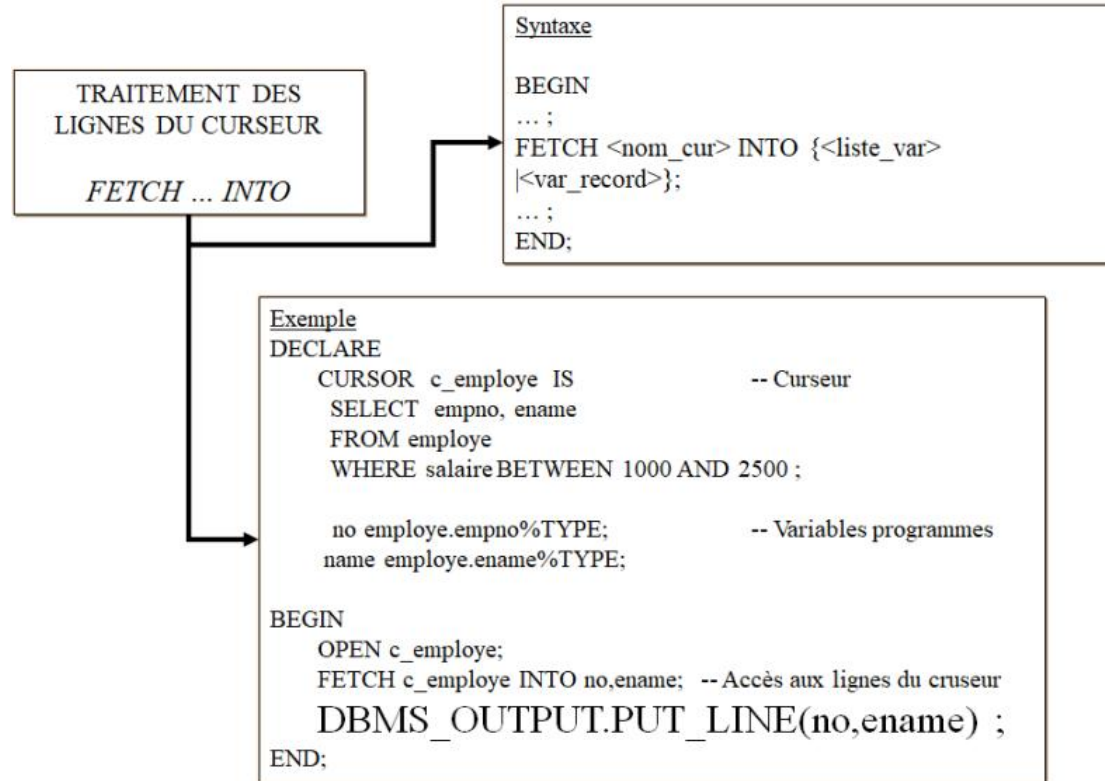
Après l'exécution du Select

- les lignes ramenées sont traitées une par une,
- la valeur de chaque colonne du Select doit être stockée dans une variable réceptrice définie dans la partie DECLARE du bloc.
- Le FETCH ramène une seule ligne à la fois,
- pour traiter n lignes il faut une boucle.

FETCH nom curseur **INTO** liste_variables ou Nom_enregistrement;



Exemple traitement





Fermeture d'un curseur

- Après le traitement des lignes, fermer le curseur :
CLOSE nom_curseur
- l'étape de fermeture permet d'effectuer la libération de la place mémoire.

```
Declare
Cursor C1 is select * from pilote where adresse='Paris';
-- variable réceptrice
unpilot pilote%rowtype;
Begin
Open C1;
Fetch c1 into unpilot ; -- Lecture 1° tuple
WHILE C1%found
loop
If unpilot.comm is not null then
insert into resultat values (unpilot.nompilot, unpilot.salpilot);
end if;
Fetch c1 into unpilot ; -- Lecture tuple suivant
end loop;
Close c1;
End ;
/
```



Récapitulatif

Instruction	Description
CURSOR nomCurseur IS requeteSelection;	Déclaration du curseur
OPEN numCurseur;	Chargement des enregistrements dans le curseur. Si il n'y a pas d'enregistrement aucune exception n'est levée
FETCH nomCurseur INTO {liste_var nom_Enreg}	Position du curseur sur l'enregistrement suivant et chargement des valeurs dans des variables ou un enregistrement.
CLOSE nomCurseur;	Fermer le curseur. Si une instruction essaye d'accéder au curseur après sa fermeture l'exception INVALID_CURSOR est levée

3

Attributs d'un curseur



Règles générales

- Ils permettent d'avoir des informations sur un curseur.
- La syntaxe utilisée: <nom_curseur>%<nom_attribut>
- Ces attributs peuvent être utilisés pour des curseurs implicites comme pour des curseurs explicites mais les résultats sont différents.

Instruction	Description
nomCurseur%ISOPEN	Retourne TRUE si le curseur est ouvert
nomCurseur%NOTFOUND	Retourne TRUE si le dernier FETCH n'a pas renvoyé d'enregistrement
nomCurseur%FOUND	Retourne TRUE si le dernier FETCH a renvoyé un enregistrement
nomCurseur%ROWCOUNT	Retourne le nombre des lignes traités jusqu'à présent



Curseur implicite

- Pas déclaré par l'utilisateur
- Utilisé par PL/SQL pour chaque opération du LMD de SQL (INSERT, UPDATE, DELETE)
- Les curseurs implicites sont tous nommés SQL

Attributs d'un curseur implicite

Attribut	Explication
SQL%ROWCOUNT	Nombre de ligne affectées par la dernière instruction LMD
SQL%FOUND	Booléen valant TRUE si la dernière instruction LMD affecte au moins un enregistrement
SQL%NOTFOUND	Booléen valant TRUE si la dernière instruction LMD n'affecte aucun enregistrement

- ISOPEN est propre au curseur explicite.
- Puisque le curseur est implicite alors on accède à ses informations par: **SQL%<nom_attribut>**



Exemple

```
BEGIN
  DELETE FROM Pilote WHERE compa='AF';
  nbDelPilote :=SQL%ROWCOUNT;
  DBMS_OUTPUT.PUT_LINE('nbre de pilote supprimé : ' || nbdelpilote);
END;
/
```

4

Parcours d'un curseur



Boucle **LOOP**

LOOP ...EXIT WHEN ... END LOOP

```
DECLARE
  CURSOR c_emp IS
    SELECT ename, sal FROM emp WHERE deptno=10 order by sal ;
  v_nom emp.ename%TYPE ;
  v_sal emp.sal%TYPE ;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_nom, v_sal ;
    EXIT WHEN c_emp%NOTFOUND ;
    IF v_sal > 2500 THEN
      UPDATE emp SET comm=comm+10 where empno=enreg.empno;

    END IF;

  END LOOP ;
  CLOSE c_emp ;
END;
```



Boucle **WHILE**

WHILE ... LOOP... END LOOP

```
DECLARE
  CURSOR c_emp IS
    SELECT ename, sal FROM emp WHERE deptno=10 order by sal ;
  enreg c_emp%ROWTYPE ;

BEGIN
  OPEN c_emp;
  FETCH c_emp INTO enreg;
  WHILE (c_emp%FOUND) LOOP
    IF enreg.sal > 2500 THEN
      UPDATE emp SET comm=comm+10 where empno=enreg.empno;

    END IF;
    FETCH c_emp INTO enreg;
  END LOOP ;
  CLOSE c_emp ;
END;
```



Boucle FOR

-1

- En utilisant la **boucle FOR** pour parcourir un curseur, on facilite la programmation : on évite les directives OPEN, FETCH et CLOSE
- La boucle s'arrête d'elle-même à la fin de l'extraction de la dernière ligne du curseur
- La variable de réception du curseur est aussi automatiquement déclarée (**%ROWTYPE du curseur**).



Boucle **FOR**

-2

```
DECLARE
CURSOR c_emp IS
SELECT * FROM emp WHERE deptno=10 order by sal ;

BEGIN
OPEN c_emp;
FETCH c_emp INTO ...;
FOR enreg IN c_emp LOOP
    IF enreg.sal > 2500 THEN
        UPDATE emp SET comm=comm+10 where empno=enreg.empno;
    END IF;
END LOOP ;
CLOSE c_emp;
END;
```



Boucle **FOR**

-3



Possibilité de déclarer le curseur dans la boucle FOR

```
DECLARE
CURSOR c_emp IS
SELECT * FROM emp WHERE deptno=10 order by sal;

BEGIN
FOR enreg IN (select * FROM emp deptno=10 order by sal) LOOP
    IF enreg.sal > 2500 THEN
        UPDATE emp SET comm=comm+10 where empno=enreg.empno;

    END IF;
END LOOP ;

END;
```



Exercice

Enoncé

sélectionner l'ensemble des employés dont le salaire ne dépasse pas 6000 et les augmenter de 500

```
DECLARE
  CURSOR SalCur IS
    SELECT * FROM EMP WHERE SAL<6000.00;
  employe EMP%ROWTYPE;
BEGIN
  OPEN SalCur;
  LOOP
    FETCH SalCur INTO employe;
    EXIT WHEN SalCur%NOTFOUND;
    UPDATE EMP
      SET SAL=SAL+500.00
      WHERE EMPNO=employe.empno;
  END LOOP;
  CLOSE SalCur ;
END;
```

```
BEGIN
  FOR employe IN SalCur
  LOOP
    UPDATE EMP
      SET SAL=6500.00 WHERE
      EMPNO=employe.empno;
  END LOOP;
END;
```


5

Curseur paramétré



Contexte général

- Un curseur peut posséder des paramètres d'entrée.
- **Objectif** : utiliser un même curseur avec des valeurs différentes, dans un même bloc PL/SQL
- Le curseur doit être fermé entre chaque utilisation de paramètres différents (sauf si on utilise « FOR » qui ferme automatiquement le curseur)

```
DECLARE
    CURSOR nomCurseur (par1 type, par2 type,...) IS ordre_select
BEGIN
    OPEN nom_c(val1, val2,...) ;
    ...
END;
```

- **Passage des valeurs des paramètres à l'ouverture du curseur**



Exemple -1

```
CURSOR enfants ( numparent NUMBER) IS  
  SELECT *  
  FROM PERSONNE  
  WHERE pere = numparent  
         OR mere = numparent ;
```



Exemple -2

Déclaration du curseur avec un paramètre.

```
DECLARE
CURSOR cur(p_dept number) IS SELECT deptno, ename FROM
emp where deptno = p_dept;
BEGIN
  FOR employe in cur(10) LOOP
    dbms_output.put_line(employe.ename);
  END LOOP;
  FOR employe in cur(20) LOOP
    dbms_output.put_line(employe.ename);
  END LOOP;
END;
/
```

Chargement et parcours du curseur en passant le paramètre 10.

Chargement et parcours du curseur en passant le paramètre 20.



Exerice complet

Enoncé

En utilisant un curseur, écrire un bloc PL/SQL permettant d'augmenter de 5% la commission d'un employé si sa fonction (job) est manager



Solution

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR c_managers IS
        SELECT numemp, nomemp, comm
        FROM employe
        WHERE UPPER(fonctionemp) = 'MANAGER'
        FOR UPDATE;
BEGIN
    FOR m IN c_managers LOOP
        IF m.comm IS NOT NULL THEN
            UPDATE employe
            SET comm = comm * 1.05
            WHERE CURRENT OF c_managers;

            DBMS_OUTPUT.PUT_LINE(m.nomemp || ' : ' || m.comm || ' → ' || (m.comm * 1.05));
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

6

Accès concurrents



Clause **FOR UPDATE**

- clause FOR UPDATE : permet de verrouiller les lignes d'une table (ou seulement les colonnes spécifiées) interrogée par un curseur dans le but de mettre à jour la table
- == > Les autres transactions ne pourront modifier les valeurs tant que le curseur n'aura pas quitté cette ligne
- Les verrous sont libérés à la fin de la transaction
- La clause WAIT / NO WAIT est utilisée avec l'instruction FOR UPDATE des curseurs pour contrôler le comportement d'attente lorsqu'une ligne est verrouillée par une autre session.

```
CURSOR nomCurseur[(paramètres)] IS  
SELECT ... FROM {nomTable | nomVue} WHERE ...  
FOR UPDATE [OF [[schéma.] {nomTable | nomVue}.]colonne [, ...]  
[ NOWAIT | WAIT entier ]
```




Exemple

```
DECLARE
CURSOR c_emp IS
SELECT * FROM scott.emp FOR UPDATE WAIT 3;
BEGIN
FOR enreg IN c_emp LOOP
    UPDATE scott.emp SET sal=1000 where empno=7369;
END LOOP ;

END;
/
```



Clause **CURRENT OF**

- permet de référencer la ligne courante d'un curseur dans une instruction UPDATE ou DELETE
- Permet de modifier/supprimer facilement la ligne courante d'un curseur
- Il faut au préalable réserver les lignes lors de la déclaration du curseur par un verrou
- Syntaxe : **WHERE CURRENT OF** nomCurseur;
- **CURRENT OF** permet d'identifier précisément la ligne sur laquelle le curseur est positionné, sans avoir à utiliser la clé primaire.

```
UPDATE employe  
SET salaire = salaire * 1.10  
WHERE numemp = emp_rec.numemp;
```

En utilisant la clé primaire (moins efficace)

```
UPDATE employe  
SET salaire = salaire * 1.10  
WHERE CURRENT OF c_employees;
```

En utilisant CURRENT OF (plus efficace)



Exemple

```
DECLARE
  CURSOR zoneModifiable IS SELECT * FROM Pilote
  FOR UPDATE OF nbHVol NOWAIT;
BEGIN
  FOR enreg IN zoneModifiable LOOP
    IF enreg.comp = 'AF' THEN
      UPDATE Pilote SET nbHVol = nbHVol + 100
      WHERE CURRENT OF zoneModifiable;
    ELSIF enreg.comp = 'SING' THEN
      UPDATE Pilote SET nbHVol = nbHVol - 100
      WHERE CURRENT OF zoneModifiable;
    ELSE
      DELETE FROM Pilote
      WHERE CURRENT OF zoneModifiable;
    END IF;
  END LOOP;

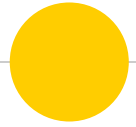
  COMMIT;

END;
/
```

Déclaration du curseur modifiable.

Mises à jour de la table Pilote par l'intermédiaire du curseur.

Validation de la transaction.



Fin du chapitre 2